



MICRONAUT CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & CLI

Installation

```
mn create-app com.example.demo
--features=data-jpa,security-jwt,graalvm
./mvnw mn:run (or gradle mnRun)
mn create-controller Hello --lang kotlin
```

All compile-time DI no Spring-like runtime scanning

04. DEPENDENCY INJECTION

Workflow

```
@Singleton / @Prototype / @RequestScope
@Inject / constructor injection
@Requires(property="feature.x") conditional ...
@Primary / @Secondary for disambiguation
```

DI processed at compile time by annotation processor
No reflection AOT proxy generation

02. MAIN APPLICATION

Architecture

```
@MicronautApplication
public class Application{
    public static void main(String[] a){
        Micronaut.run(Application.class, a);
    }
}
```

Starts in <100ms; tiny memory footprint

05. CONFIGURATION

CRUD API

```
application.yml / application-prod.yml
@ConfigurationProperties("app")
public class AppConfig {
    private String name; // maps to app.name
}
@Value("${greeting:Hello}") String greeting;
```

Env vars override properties automatically

03. CONTROLLERS

YAML / Config

```
@Controller("/api/users")
public class UserController {
    @Get("/{id}") @Produces(APPLICATION_JSON)
    public HttpResponse<User> get(@PathVariable L...
    @Post @Consumes(APPLICATION_JSON)
    public HttpResponse<User> create(@Body User u...
```

06. DATA REPOSITORIES

Integration

```
@Repository
interface UserRepo extends CrudRepository<Use...
List<User> findByEmail(String email);
@Query("SELECT * FROM users WHERE active=true...
List<User> findActive();
}
```

Micronaut Data generates SQL at compile time



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. AOP & INTERCEPTORS

Hardening

```
@Around
@Type(LogInterceptor.class)
public @interface Logged {}
@Singleton @InterceptorBean(Logged.class)
class LogInterceptor implements MethodInterce...
Object intercept(MethodInvocationContext ctx)...
}
```

10. TESTING

Unit Tests

```
@MicronautTest
class UserControllerTest {
@Inject @Client("/") HttpClient client;
@Test void testGet(){
var resp = client.toBlocking().exchange("/use...
assertEquals(200, resp.status().getStatusCode());
}
}
```

08. DECLARATIVE HTTP CLIENT

Observability

```
@Client("http://user-service")
interface UserClient {
@Get("/users/{id}")
User getUser(@PathVariable Long id);
}
@Inject UserClient client;
Compile-time generated no reflection, native-compatible
```

11. COMMON ANNOTATIONS

Commands

```
@MicronautApplication @Controller @Get @Post ...
@Singleton @Inject @Value @ConfigurationPrope...
@Client @Body @PathVariable @QueryValue
@Introspected // for reflection-less seriali...
@Requires // conditional beans
@Validated @NotNull @Min @Max bean validation
```

09. COMPILE-TIME DI / AOT

Optimization

Annotation processors run during compilation
DI metadata baked into bytecode (not reflection)
No runtime classpath scanning
Much faster startup than Spring
GraalVM native: mn:nativeCompile

```
./target/myapp starts in <20ms
```

12. COMMON GOTCHAS

Warning

@Introspected needed on data classes for native image
Circular deps: Micronaut fails at compile time (good!)
AOP only works on injected beans, not new'd instances
HTTP client must match server's content-type exactly
Config refresh requires restart no @RefreshScope